

" به نام خدا "



دانشکده فنی مهندسی

رشته مهندسی کامپیوتر

گرایش هوش مصنوعی و نرم افزار

عنوان :

گزارش سوم پروژه نهایی مباحث ویژه

دانشجویان :

محمد رضا باقرنژاد میری، یاسمن دادفر، کیارش رحمانی، معصومه مختاری

استاد :

دکتر مهدیه قاسمی

فهرست

۱.مقدمه.....	۳
۲. روش انجام کار	۵
۱-۲. ایجاد حالت پایه ترتیبی	۶
۲-۲. پیاده‌سازی بهینه‌سازی سطح اول: Batched Inference و Vectorization	۶
۳-۲. پیاده‌سازی روش جایگزین برای فشرده‌سازی مدل	۷
۴-۲. بررسی محدودیت Quantization و دلیل کنار گذاشتن آن	۸
۵-۲. رویکرد جایگزین : Vocabulary Pruning	۱۰
۶-۲. نتایج تجربی و تحلیل عملکرد	۱۱
۷-۲. توسعه لایه سرویس‌دهی مدل با FastAPI	۱۳
۸-۲. تعریف ساختار داده‌های ورودی و خروجی با Pydantic	۱۴
۹-۲. اجرای سرویس با Uvicorn و آماده‌سازی مسیر عملیاتی API	۱۵
۱۰-۲. پیاده‌سازی Micro-Batching در لایه سرویس.....	۱۵
۱۱-۲. افزودن مسیر inference مبتنی بر ONNX Runtime	۱۶
۱۲-۲. Dockerize کردن سرویس API	۱۷
۱۳-۲. پایش منابع و Benchmarking سرویس	۱۸
۱۴-۲. تحلیل عملکرد Dynamic Quantization و بهبود سرعت Inference	۱۸

١. مقدمه

در فاز سوم پروژه، تمرکز تیم روی تحلیل عملکرد مدل پایه در زمان استنتاج و بررسی روش‌های بهینه‌سازی سرعت پیش‌بینی قرار گرفت. پس از آنکه در فازهای قبلی، Pipeline داده، آموزش مدل و ثبت نتایج با استفاده از ابزارهایی مانند MLflow آماده شده بود، در این مرحله مدل از دیدگاه اجرای عملیاتی بررسی شد؛ یعنی هدف فقط ارزیابی دقت مدل نبود، بلکه زمان پاسخ‌گویی، نحوه اجرای inference و تأثیر روش‌های بهینه‌سازی بر اندازه مدل و دقت نیز مورد توجه قرار گرفت.

مدل مورد بررسی در این فاز، همان مدل پایه کلاسیک پروژه شامل Logistic و TF-IDF Vectorizer و Regression بود. برای تحلیل دقیق‌تر عملکرد این مدل، ابتدا اجرای ترتیبی آن بررسی شد؛ به این صورت که نمونه‌های متنی یکی‌یکی به مدل داده شدند تا حالت پایه و بدون بهینه‌سازی شبیه‌سازی شود. این حالت به عنوان معیار مرجع برای مقایسه با روش‌های بعدی در نظر گرفته شد.

در ادامه، تیم روش Batched Inference را پیاده‌سازی و بررسی کرد. در این روش، به جای ارسال تک‌تک نمونه‌ها به مدل، مجموعه‌ای از متن‌ها به صورت یکجا به Pipeline داده شد تا از قابلیت‌های پردازش برداری در NumPy، Scikit-Learn و عملیات ماتریسی استفاده شود. هدف از این کار، کاهش سرشار ناشی از حلقه‌های ترتیبی و افزایش سرعت کلی inference بود.

همچنین در این فاز، امکان استفاده از Quantization برای مدل پایه بررسی شد. با توجه به اینکه مدل مورد استفاده از نوع کلاسیک و مبتنی بر Scikit-Learn بود، مشخص شد که Quantization به شکل رایج آن برای این ساختار مناسب نیست. به همین دلیل، تیم به جای آن از روش Vocabulary Pruning استفاده کرد تا با محدود کردن فضای ویژگی‌های TF-IDF، اندازه مدل و هزینه محاسباتی کاهش یابد. در پایان، نتایج روش‌های مختلف در قالب یک ماتریس مقایسه‌ای شامل اندازه مدل، latency و accuracy تحلیل شد.

۲. روش انجام کار

۲-۱. ایجاد حالت پایه ترتیبی

فرآیند بهینه‌سازی مدل پایه در این فاز به صورت مرحله‌ای و در فایل `optimize_baseline.py` پیاده‌سازی شد. در اولین مرحله، هدف این بود که عملکرد مدل بدون اعمال هیچ‌گونه بهینه‌سازی اندازه‌گیری شود تا یک نقطه مرجع برای مقایسه با روش‌های بعدی به دست آید. به همین دلیل، ابتدا حالت `Sequential Baseline` برای مدل پایه ایجاد شد.

در این مرحله، مدل آموزش‌دیده و بهینه‌سازی‌نشده با نام `baseline_model.pkl` از حافظه بارگذاری شد. این مدل همان `Pipeline` پایه شامل `TF-IDF Vectorizer` و `Logistic Regression` بود که در مراحل قبلی پروژه آموزش داده شده بود. پس از بارگذاری مدل، فرآیند `inference` به صورت ترتیبی شبیه‌سازی شد؛ به این معنا که نمونه‌های متنی موجود در مجموعه اعتبارسنجی، یکی‌یکی وارد `Pipeline` شدند و مدل برای هر نمونه به صورت جداگانه خروجی پیش‌بینی تولید کرد.

این شیوه اجرا، حالتی ساده و بدون بهینه‌سازی از عملکرد مدل را نشان می‌دهد و می‌تواند مشابه وضعیتی باشد که در آن یک `API` به ازای هر درخواست کاربر، فقط یک متن را دریافت و پردازش می‌کند. در چنین حالتی، سربار ناشی از اجرای مکرر حلقه، فراخوانی‌های جداگانه مدل و پردازش تک‌نمونه‌ای در زمان پاسخ‌گویی تأثیر مستقیم دارد. پس از اجرای این مرحله، معیارهایی مانند زمان کل `inference`، میانگین `latency` برای هر نمونه بر حسب میلی‌ثانیه و `accuracy` مدل استخراج شدند تا مبنای تحلیل اثر روش‌های بهینه‌سازی بعدی قرار گیرند.

۲-۲. پیاده‌سازی بهینه‌سازی سطح اول: `Batched Inference` و `Vectorization`

پس از اندازه‌گیری عملکرد مدل در حالت ترتیبی، مرحله دوم آزمایش با هدف کاهش زمان استنتاج و حذف سربار ناشی از اجرای نمونه‌به‌نمونه انجام شد. در این مرحله، به جای آنکه هر متن به صورت جداگانه وارد `Pipeline` شود، کل مجموعه متن‌های ارزیابی به صورت یکجا به مدل داده شد. این تغییر در فایل

optimize_baseline.py اعمال شد و حلقه ترتیبی قبلی با فراخوانی مستقیم pipeline.predict(texts) جایگزین گردید.

دلیل اصلی این تغییر، استفاده بهتر از قابلیت‌های پردازش برداری در Scikit-Learn و کتابخانه‌های زیرساختی مانند NumPy و BLAS بود. در حالت ترتیبی، هر نمونه به صورت مستقل از مراحل تبدیل متن به بردار TF-IDF و سپس محاسبه خروجی Logistic Regression عبور می‌کند. اما در حالت batched، محاسبات مربوط به استخراج ویژگی‌ها و ضرب‌های ماتریسی برای مجموعه‌ای از نمونه‌ها به شکل یکپارچه انجام می‌شود. این ساختار باعث می‌شود بخش بیشتری از توان پردازشی CPU و بهینه‌سازی‌های سطح پایین کتابخانه‌های عددی مورد استفاده قرار گیرد.

در این مرحله، خود مدل و وزن‌های آموزش‌دیده تغییری نکردند و تنها نحوه ارسال داده‌ها به مدل و اجرای پیش‌بینی اصلاح شد. بنابراین، انتظار نمی‌رفت دقت مدل نسبت به حالت پایه تغییر کند. هدف اصلی این مرحله، بهینه‌سازی مسیر اجرای inference و کاهش میانگین زمان پیش‌بینی برای هر نمونه بود، بدون آنکه ساختار مدل یا پارامترهای یادگرفته‌شده تغییر داده شوند.

۲-۳. پیاده‌سازی روش جایگزین برای فشردن مدل

در مرحله سوم، تمرکز تیم روی کاهش اندازه مدل و بررسی اثر آن بر زمان اجرای inference قرار گرفت. از آنجا که مدل پایه پروژه یک Pipeline کلاسیک مبتنی بر TF-IDF Vectorizer و Logistic Regression بود، روش فشردن مدل باید با ساختار این نوع مدل سازگار می‌بود. به همین دلیل، یک نسخه فشرده‌شده از مدل با نام baseline_pruned.pkl ایجاد شد تا اثر کاهش ساختاری فضای ویژگی‌ها بر اندازه فایل مدل، زمان پاسخ‌گویی و دقت بررسی شود.

در این نسخه، به جای تغییر مستقیم وزن‌های مدل یا استفاده از روش‌هایی که بیشتر برای مدل‌های عمیق کاربرد دارند، فشرده‌سازی از طریق محدود کردن فضای واژگان و کاهش تعداد ویژگی‌های ورودی انجام شد. این کار باعث شد ابعاد بردارهای TF-IDF و در نتیجه اندازه ماتریس ضرایب مدل Logistic Regression کاهش پیدا کند. پس از ایجاد مدل فشرده‌شده، فایل `baseline_pruned.pkl` از حافظه بارگذاری شد و عملکرد آن در حالت `batched` ارزیابی گردید.

ارزیابی مدل فشرده‌شده در حالت `batched` باعث شد اثر واقعی کاهش فضای ویژگی‌ها بدون غلبه سربار حلقه‌های ترتیبی بررسی شود. در این مرحله، معیارهایی مانند اندازه فایل مدل، میانگین `latency` برای هر نمونه و `accuracy` اندازه‌گیری شدند. در پایان، نتایج حاصل از سه وضعیت مختلف، یعنی مدل ترتیبی پایه، مدل `batched` و مدل فشرده‌شده، در قالب یک ماتریس مقایسه‌ای ساختاریافته جمع‌آوری شد تا تأثیر هر روش بر اندازه مدل، سرعت پاسخ‌گویی و دقت نهایی مشخص شود.

۲-۴. بررسی محدودیت Quantization و دلیل کنار گذاشتن آن

در ادامه فرآیند بهینه‌سازی، روش Quantization به عنوان یکی از گزینه‌های رایج برای کاهش اندازه مدل و افزایش سرعت `inference` بررسی شد. ایده اصلی در Quantization این است که وزن‌های مدل از مقادیر اعشاری با دقت بالا، مانند اعداد ۳۲ بیتی، به نمایش‌های عددی سبک‌تر مانند اعداد صحیح ۸ بیتی تبدیل شوند. این روش معمولاً در مدل‌های یادگیری عمیق و چارچوب‌هایی مانند TensorFlow، PyTorch یا ONNX Runtime کاربرد زیادی دارد، زیرا این چارچوب‌ها از موتورهای اجرایی و Kernel‌های مخصوص برای محاسبات `low-bit` پشتیبانی می‌کنند.

با این حال، در پروژه ما مدل پایه از نوع کلاسیک و مبتنی بر Scikit-Learn بود و شامل TF-IDF Vectorizer و Logistic Regression می‌شد. به همین دلیل، پس از بررسی ساختار مدل مشخص شد

که اعمال Quantization به شکل استاندارد برای این Pipeline مناسب نیست Scikit-Learn. به طور معمول روی آرایه‌ها و ماتریس‌های NumPy و SciPy اجرا می‌شود و محاسبات آن برای دقت‌های عددی رایج مانند float32 یا float64 بهینه شده است. برخلاف چارچوب‌های یادگیری عمیق، این کتابخانه یک runtime اختصاصی برای اجرای ضرب‌های ماتریسی با نوع داده‌های qint8 یا شبیه‌سازی کامل محاسبات ۸ بیتی در اختیار مدل قرار نمی‌دهد.

مسئله مهم دیگر به ماهیت ویژگی‌های تولیدشده توسط TF-IDF مربوط بود. خروجی TF-IDF معمولاً یک ماتریس بسیار بزرگ و sparse است که در آن بسیاری از ویژگی‌ها مقدار صفر دارند و تعداد زیادی از مقادیر غیرصفر نیز بسیار کوچک و اعشاری هستند. این مقادیر کوچک نشان‌دهنده وزن واژه‌ها و اهمیت نسبی آن‌ها در متن هستند. اگر این مقادیر به صورت مستقیم به فضای عددی ۸ بیتی تبدیل شوند، بخش زیادی از جزئیات عددی آن‌ها از بین می‌رود و بسیاری از وزن‌های کوچک ممکن است به صفر تبدیل شوند.

از بین رفتن این مقادیر کوچک می‌تواند باعث تخریب جدی نمایش معنایی متن شود؛ زیرا مدل دیگر نمی‌تواند تفاوت ظریف میان واژه‌ها و اهمیت نسبی آن‌ها را به درستی تشخیص دهد. در چنین شرایطی، ماتریس ویژگی‌ها تا حد زیادی فرو می‌ریزد و دقت مدل ممکن است به شکل قابل توجهی کاهش پیدا کند. بنابراین، تیم تصمیم گرفت Quantization را برای این مدل اجرا نکند و به جای آن از روشی استفاده کند که با ساختار TF-IDF و Logistic Regression سازگارتر باشد.

در نتیجه، کنار گذاشتن Quantization یک تصمیم صرفاً اجرایی نبود، بلکه بر اساس محدودیت‌های معماری مدل و نوع داده‌های تولیدشده توسط TF-IDF انجام شد. این بررسی نشان داد که روش‌های بهینه‌سازی باید متناسب با نوع مدل انتخاب شوند و تکنیکی که برای مدل‌های عمیق مناسب است، الزاماً برای Pipeline‌های کلاسیک یادگیری ماشین نتیجه مطلوبی ایجاد نمی‌کند.

۵-۲. رویکرد جایگزین : Vocabulary Pruning

پس از بررسی محدودیت‌های Quantization برای مدل پایه، تیم از روش Vocabulary Pruning به عنوان رویکرد جایگزین برای فشرده‌سازی مدل استفاده کرد. هدف از این تصمیم آن بود که بدون تغییر ناسازگار در ساختار عددی Pipeline، اندازه مدل و هزینه محاسباتی inference کاهش یابد. از آنجا که مدل پروژه بر پایه TF-IDF Vectorizer و Logistic Regression طراحی شده بود، مناسب‌ترین محل برای کاهش حجم مدل، فضای ویژگی‌های متنی تولیدشده توسط Vectorizer بود.

در حالت عادی، TF-IDF Vectorizer می‌تواند تعداد زیادی از واژه‌ها و n-gram‌های موجود در داده آموزشی را به عنوان ویژگی نگه دارد. این موضوع باعث می‌شود فضای ویژگی‌ها بسیار بزرگ شود و در نتیجه، ماتریس وزن‌های داخلی مدل Logistic Regression نیز ابعاد بزرگ‌تری پیدا کند. در روش Vocabulary Pruning، این فضای ویژگی به صورت کنترل‌شده محدود شد. برای این کار، پارامتر max_features در TF-IDF Vectorizer تنظیم شد تا فقط تعداد مشخصی از مهم‌ترین ویژگی‌ها در مدل باقی بمانند و واژه‌ها یا n-gram‌های کم‌اثر و بسیار نادر حذف شوند.

این روش از نظر عملی باعث کاهش عرض بردارهای TF-IDF شد؛ یعنی هر متن به جای نمایش در یک فضای بسیار بزرگ، در یک فضای کوچک‌تر و کنترل‌شده‌تر نمایش داده شد. در نتیجه، تعداد ضرایب مورد نیاز در مدل Logistic Regression نیز کاهش پیدا کرد. کاهش تعداد ویژگی‌ها مستقیماً روی اندازه فایل ذخیره‌شده مدل اثر گذاشت و باعث شد نسخه pruned مدل با حجم کمتری نسبت به نسخه اصلی ذخیره شود.

مزیت دیگر این روش، کاهش هزینه محاسباتی در زمان inference بود. وقتی تعداد ویژگی‌های ورودی کمتر باشد، عملیات ضرب داخلی میان بردار TF-IDF و ضرایب مدل نیز سبک‌تر انجام می‌شود. این موضوع به ویژه در حالت batched اهمیت بیشتری دارد، زیرا مدل باید هم‌زمان روی مجموعه‌ای از نمونه‌ها عملیات ماتریسی انجام

دهد. بنابراین، Vocabulary Pruning علاوه بر کاهش اندازه فایل .pkl، می‌تواند زمان محاسباتی لازم برای پیش‌بینی دسته‌ای را نیز کاهش دهد.

در مجموع، Vocabulary Pruning برای این مدل نقش جایگزین عملی Quantization را ایفا کرد. این روش بدون آنکه ساختار عددی ویژگی‌های TF-IDF را تخریب کند، بخشی از اهداف فشرده‌سازی مدل را محقق کرد؛ یعنی کاهش حجم مدل، کاهش سربار محاسباتی و حفظ دقت در سطح قابل قبول. به همین دلیل، برای Pipeline کلاسیک مورد استفاده در این پروژه، این روش سازگارتر و مطمئن‌تر از Quantization در نظر گرفته شد.

۲-۶. نتایج تجربی و تحلیل عملکرد

پس از اجرای مراحل مختلف بهینه‌سازی، نتایج به دست آمده در قالب یک ماتریس مقایسه‌ای ساختاریافته جمع‌آوری شد. هدف از این ماتریس آن بود که اثر هر روش بهینه‌سازی بر سه معیار اصلی، یعنی اندازه مدل، میانگین زمان پاسخ‌گویی برای هر نمونه و دقت مدل، به صورت یکپارچه قابل بررسی باشد. سه وضعیت اصلی مورد مقایسه شامل اجرای ترتیبی مدل پایه، اجرای batched به عنوان بهینه‌سازی سطح اول، و نسخه فشرده‌شده مدل با استفاده از Vocabulary Pruning بودند.

نتایج نشان داد که تغییر مسیر اجرا از حلقه ترتیبی به پردازش دسته‌ای، بیشترین تأثیر را بر کاهش latency داشته است. در حالت Sequential Baseline، هر نمونه متنی به صورت جداگانه وارد مدل می‌شد و همین موضوع باعث افزایش سربار ناشی از فراخوانی‌های مکرر Pipeline و اجرای تک‌نمونه‌ای می‌گردید. اما در حالت Batched Inference، کل مجموعه داده ارزیابی به صورت یکجا به مدل داده شد و محاسبات TF-IDF و Logistic Regression به شکل برداری انجام گرفت. این تغییر باعث شد میانگین زمان پیش‌بینی برای هر نمونه به طور قابل توجهی کاهش پیدا کند.

نکته مهم در مورد بهینه‌سازی سطح اول این بود که این روش فقط نحوه اجرای inference را تغییر داد و هیچ تغییری در ساختار مدل، وزن‌های آموزش‌دیده یا فضای ویژگی‌ها ایجاد نکرد. بنابراین، دقت مدل نسبت به حالت پایه حفظ شد. این نتیجه نشان می‌دهد که در مدل‌های کلاسیک یادگیری ماشین، بخش قابل توجهی از تأخیر می‌تواند ناشی از سربار نرم‌افزاری و اجرای ترتیبی باشد، نه الزاماً پیچیدگی خود مدل.

در مرحله بعد، مدل فشرده‌شده با استفاده از Vocabulary Pruning مورد ارزیابی قرار گرفت. این روش باعث کاهش اندازه فایل ذخیره‌شده مدل شد، زیرا تعداد ویژگی‌های تولیدشده توسط TF-IDF محدودتر شد و در نتیجه ابعاد ماتریس ضرایب Logistic Regression کاهش یافت. از نظر زمان اجرا نیز کاهش فضای ویژگی‌ها باعث سبک‌تر شدن محاسبات ماتریسی در حالت batched شد. با این حال، از آنجا که بخشی از واژه‌ها و- n gram‌های کم‌اهمیت حذف شدند، احتمال افت جزئی در دقت وجود داشت. نتایج نشان داد که این افت در سطح ناچیز باقی مانده و مدل فشرده‌شده همچنان عملکرد قابل قبولی داشته است.

برای ارزیابی عملی اثر هر روش بهینه‌سازی روی مدل پایه، آزمایش‌ها روی کل مجموعه داده اعتبارسنجی اجرا شد. جدول زیر نتایج زمان پاسخ‌گویی، اندازه مدل و دقت پیش‌بینی را برای سه وضعیت Sequential، Batched و Vocabulary Pruned نشان می‌دهد.

جدول ۱-۲ نتایج زمان پاسخ‌گویی برای هر سه وضعیت

Optimization State	Model	Size (MB)	Avg Latency / Sample	Validation Accuracy	Size Reduction
Sequential Baseline		۰.۴۴۰	۰.۶۸۰ ms	۰.۸۷۷۶	۰٪ (Baseline)
Batched Inference (L1)		۰.۴۴۰	۰.۱۲۲ ms	۰.۸۷۷۶	۰٪ (Baseline)
Vocabulary Pruned (L2)		۰.۰۴۴	۰.۲۰۵ ms	۰.۸۸۲۱	-۹۰.۱٪

به طور کلی، ماتریس عملکرد نشان داد که Batched Inference بهترین روش برای کاهش latency بدون تغییر دقت مدل بود، در حالی که Vocabulary Pruning بیشتر برای کاهش اندازه مدل و کاهش نسبی هزینه محاسباتی مفید واقع شد. این تحلیل نشان می‌دهد که برای Pipeline های کلاسیک مبتنی بر TF-IDF و Logistic Regression، بهینه‌سازی مسیر اجرا و کنترل اندازه فضای ویژگی‌ها می‌تواند بدون نیاز به تغییر معماری مدل، به بهبود عملی عملکرد inference کمک کند.

۲-۷. توسعه لایه سرویس‌دهی مدل با FastAPI

پس از آماده‌سازی و بررسی مدل از نظر عملکرد استنتاج، بخش سرویس‌دهی مدل در پروژه پیاده‌سازی شد تا مدل بتواند از طریق یک API قابل استفاده باشد. در این مرحله، FastAPI به عنوان چارچوب اصلی توسعه سرویس انتخاب شد و endpoint های لازم برای ارتباط با مدل در ساختار پروژه قرار گرفتند. هدف از این بخش، جدا کردن استفاده از مدل از اجرای مستقیم اسکریپت‌های آموزشی و فراهم کردن یک مسیر استاندارد برای دریافت درخواست و برگرداندن پاسخ بود.

در این ساختار، API نقش واسطه میان کاربر و مدل را ایفا می‌کند. کاربر متن ورودی را به سرویس ارسال می‌کند و سرویس پس از پردازش درخواست، نتیجه پیش‌بینی مدل را در قالب پاسخ JSON برمی‌گرداند. علاوه بر endpoint پیش‌بینی، مسیرهایی برای بررسی سلامت سرویس و دریافت اطلاعات مربوط به وضعیت مدل نیز در نظر گرفته شد تا امکان تست و پایش اولیه سرویس فراهم باشد. شکل ۲-۱ نمونه‌ای از تست endpoint پیش‌بینی API و پاسخ JSON بازگشتی را نشان می‌دهد.

```
user@DESKTOP-6090D4R:/mnt/c/Users/User/Desktop/New folder (2)$ curl -s --max-time 15 -X POST "http://127.0.0.1:8081/predict?variant=batched" -H "Content-Type: application/json" -d '{"text": "This movie was absolutely wonderful!"}'
| python3 -m json.tool
{
  "sentiment": "positive",
  "confidence": 0.8692731068731929,
  "probabilities": {
    "negative": 0.13072689312680708,
    "neutral": 0.0,
    "positive": 0.8692731068731929
  },
  "model_variant": "batched"
}
```

شکل ۱-۲ نمونه تست endpoint پیش‌بینی API و دریافت خروجی تحلیل احساسات

استفاده از FastAPI همچنین باعث شد مستندات API به صورت خودکار تولید شود. این مستندات، ساختار درخواست‌ها و پاسخ‌ها را نمایش می‌دهند و فرآیند تست endpointها را ساده‌تر می‌کنند. در نتیجه، مدل پروژه از حالت یک خروجی آموزشی خارج شد و در قالب یک سرویس قابل فراخوانی و قابل تست قرار گرفت.

۸-۲. تعریف ساختار داده‌های ورودی و خروجی با Pydantic

برای کنترل کیفیت داده‌هایی که وارد سرویس می‌شوند، در لایه API از Pydantic استفاده شد. در این بخش، schema های مشخصی برای request و response تعریف شدند تا API فقط ورودی‌هایی با قالب معتبر را بپذیرد. این موضوع باعث شد پیش از ارسال داده به مدل، ساختار ورودی بررسی شود و خطاهای مربوط به داده‌های ناقص یا نامعتبر در سطح سرویس مدیریت شوند.

تعریف schemaها همچنین باعث شد خروجی مدل به شکل یکپارچه و قابل پیش‌بینی به کاربر برگردانده شود. به جای آنکه پاسخ سرویس به صورت پراکنده یا بدون ساختار مشخص تولید شود، خروجی‌ها در قالب مدل‌های داده‌ای تعریف‌شده برگردانده شدند. این کار هم خوانایی API را افزایش داد و هم امکان مستندسازی خودکار آن را دقیق‌تر کرد.

در مجموع، Pydantic در این بخش نقش یک لایه کنترلی میان ورودی کاربر و مدل را داشت. این لایه باعث شد API از نظر قالب داده، اعتبارسنجی و پاسخ‌دهی ساختارمندتر شود و برای استفاده در محیط عملیاتی قابل اعتمادتر باشد.

۲-۹. اجرای سرویس با Uvicorn و آماده‌سازی مسیر عملیاتی API

برای اجرای برنامه FastAPI، از Uvicorn به عنوان سرور ASGI استفاده شد. این سرور وظیفه اجرای برنامه، دریافت درخواست‌های HTTP و ارسال آن‌ها به endpointهای تعریف‌شده در FastAPI را بر عهده داشت. به این ترتیب، API پروژه روی یک host و port مشخص قابل اجرا شد و امکان ارسال درخواست‌های واقعی به سرویس فراهم گردید.

استفاده از Uvicorn باعث شد لایه سرویس‌دهی پروژه از نظر اجرایی به ساختار رایج سرویس‌های Python نزدیک شود. در این حالت، مدل دیگر فقط از طریق اجرای مستقیم فایل‌های پایتون استفاده نمی‌شود، بلکه در قالب یک برنامه وب در دسترس قرار می‌گیرد. این موضوع برای اتصال سرویس به سایر اجزای پروژه، تست محلی، و اجرای آن در Docker اهمیت داشت.

این بخش بیشتر به آماده‌سازی مسیر اجرای API مربوط بود و تکمیل‌کننده پیاده‌سازی FastAPI محسوب می‌شد. به کمک Uvicorn، سرویس مدل به صورت یک برنامه قابل اجرا و قابل فراخوانی آماده شد.

۲-۱۰. پیاده‌سازی Micro-Batching در لایه سرویس

در کنار امکان پیش‌بینی مستقیم، ساختار micro-batching نیز در لایه سرویس‌دهی پروژه پیاده‌سازی شد. هدف از این بخش، مدیریت بهتر درخواست‌هایی بود که به صورت جداگانه به API ارسال می‌شوند، اما می‌توانند

در پشت صحنه به شکل گروهی پردازش شوند. در این روش، درخواست‌های ورودی برای مدت کوتاهی در صف قرار می‌گیرند و سپس چند درخواست با هم به مدل ارسال می‌شوند.

این روش با استفاده از سازوکارهای `async` در پایتون پیاده‌سازی شد و پارامترهایی مانند حداکثر اندازه `batch` و زمان انتظار برای تشکیل `batch` در تنظیمات سرویس قرار گرفتند. به این ترتیب، کلاینت همچنان می‌تواند درخواست تک‌متنی ارسال کند، اما سرویس در لایه داخلی خود چند درخواست نزدیک به هم را جمع می‌کند. تفاوت این بخش با آزمایش‌های قبلی در این است که `batching` در اینجا فقط یک سناریوی `benchmark` نیست، بلکه به عنوان بخشی از منطق سرویس‌دهی API استفاده شده است. بنابراین، `micro-batching` پلی میان درخواست‌های تک‌نمونه‌ای کاربران و اجرای کارآمدتر مدل در سمت سرور ایجاد می‌کند.

۲-۱۱. افزودن مسیر `inference` مبتنی بر `ONNX Runtime`

در این فاز، علاوه بر مسیر اجرای مدل ذخیره‌شده به صورت `pkl`، مسیر جداگانه‌ای برای اجرای مدل با استفاده از `ONNX` نیز در پروژه در نظر گرفته شد. برای این کار، `Pipeline` مبتنی بر `Scikit-Learn` به فرمت `ONNX` تبدیل شد تا امکان اجرای آن در یک `runtime` مستقل و مناسب برای `inference` فراهم شود. `ONNX Runtime` به عنوان موتور اجرای این نسخه مورد استفاده قرار گرفت. این مسیر به پروژه اجازه می‌دهد که مدل در قالبی قابل حمل‌تر اجرا شود و وابستگی مستقیم آن به اجرای سنتی `Scikit-Learn` در زمان سرویس‌دهی کاهش یابد. همچنین، ساختار پروژه امکان استفاده از نسخه `quantized` مدل `ONNX` را نیز فراهم کرد تا یک مسیر بهینه‌شده جداگانه برای `inference` در دسترس باشد.

نکته مهم این است که این بخش با تصمیم قبلی درباره عدم اجرای `Quantization` مستقیم روی `Pipeline` اصلی تفاوت دارد. در اینجا مدل ابتدا به قالب `ONNX` منتقل شده و سپس از قابلیت‌های `Runtime` مربوط به

آن استفاده شده است. بنابراین، ONNX Runtime به عنوان یک مسیر اجرایی مستقل برای سرویس‌دهی مدل در نظر گرفته شد، نه صرفاً ادامه همان Pipeline اولیه.

۱۲-۲. Dockerize کردن سرویس API

برای اجرای قابل بازتولید سرویس در محیط‌های مختلف، بخش API پروژه نیز Dockerize شد. در این مرحله، وابستگی‌های مربوط به سرویس‌دهی مدل، شامل FastAPI، Uvicorn، کتابخانه‌های inference و تنظیمات لازم، در قالب یک Docker image مشخص قرار گرفتند. این کار باعث شد اجرای API به وضعیت نصب پکیج‌ها روی سیستم محلی وابسته نباشد.

در ساختار پروژه، برای لایه سرویس‌دهی از Dockerfile.api استفاده شد. این Dockerfile با هدف ساخت یک image مناسب برای اجرای API طراحی شد و وابستگی‌های سرویس را از وابستگی‌های مربوط به آموزش مدل جدا کرد. این جداسازی باعث سبک‌تر شدن محیط اجرای API و شفاف‌تر شدن نقش هر بخش در پروژه شد.

همچنین با استفاده از Docker Compose امکان اجرای هماهنگ چند سرویس پروژه فراهم شد. در نتیجه، API می‌توانست در کنار اجزای دیگر مانند MLflow اجرا شود و مدل مورد نیاز خود را از مسیرهای تعریف‌شده بارگذاری کند. این مرحله باعث شد لایه سرویس‌دهی پروژه به شکل قابل حمل‌تر و نزدیک‌تر به محیط production آماده شود.

۲-۱۳. پایش منابع و Benchmarking سرویس

برای بررسی عملکرد سرویس و مقایسه مسیرهای مختلف inference، از ابزارهای benchmarking در پروژه استفاده شد. در این بخش، هدف تنها بررسی دقت مدل نبود، بلکه معیارهایی مانند زمان پاسخ‌گویی، مصرف حافظه و رفتار سرویس هنگام اجرای inference نیز مورد توجه قرار گرفتند.

برای اندازه‌گیری مصرف منابع سیستم، از psutil استفاده شد. این ابزار امکان مشاهده وضعیت مصرف حافظه و منابع پردازشی را در زمان اجرای سناریوهای مختلف فراهم کرد. در کنار آن، اسکریپت‌های benchmark پروژه برای اجرای تست‌های عملکردی و جمع‌آوری نتایج مورد استفاده قرار گرفتند.

خروجی این بخش به تیم کمک کرد تا وضعیت سرویس را نه فقط از نظر عملکرد مدل، بلکه از نظر هزینه اجرایی و مصرف منابع نیز ارزیابی کند. بنابراین، Benchmarking در این مرحله نقش مکملی برای تحلیل‌های قبلی داشت و دید عملی‌تری نسبت به رفتار مدل در زمان سرویس‌دهی ارائه داد.

۲-۱۴. تحلیل عملکرد Dynamic Quantization و بهبود سرعت

Inference

پس از ایجاد مسیر ONNX برای اجرای مدل DistilBERT، مرحله بعدی تحلیل عملی اثر Dynamic Quantization و Batching روی اندازه مدل، سرعت inference و دقت آن بود. این بررسی به تیم اجازه داد تا مزایای بهینه‌سازی حافظه و محاسباتی را به شکل ملموس مشاهده کند.

نتایج نشان داد که اعمال Dynamic Quantization همراه با پردازش دسته‌ای باعث کاهش چشمگیر اندازه مدل از حدود ۲,۵۵۵ مگابایت به ۱۳۲ مگابایت شد. این کاهش بیش از ۹۴ درصد حجم ذخیره‌سازی را فراهم

کرد، زیرا تمام داده‌های اضافه مربوط به آموزش، شامل shard ها، optimizer.pt و لاگ‌های trainer حذف شدند و تنها هسته عملیات پیش‌بینی باقی ماند. این فشرده‌سازی به کاهش مصرف حافظه و افزایش کارایی سرویس کمک شایانی کرد.

در کنار کاهش حجم مدل، سرعت پیش‌بینی نیز بهبود قابل توجهی پیدا کرد. میانگین latency برای هر نمونه از حدود ۲۸۲ میلی‌ثانیه در حالت Sequential Baseline به ۱۰۲.۷۸ میلی‌ثانیه در حالت Dynamic Quantized + Batch کاهش یافت که تقریباً ۲.۷۴ برابر سرعت بیشتر فراهم کرد. این بهبود از ترکیب دو لایه بهینه‌سازی حاصل شد: اول، حذف حلقه‌های ترتیبی و اجرای inference در قالب batch با اندازه ۳۲ نمونه که امکان بهره‌گیری از پردازش برداری CPU و کاهش سربار tokenization را فراهم می‌کند، و دوم، تبدیل وزن‌های floating-point لایه‌های Linear به qint8 که موجب کاهش فشار حافظه و misses شد.

با وجود این مزایا، دقت مدل تحت تأثیر Quantization کاهش پیدا کرد؛ accuracy از ۸۹.۶۰٪ به ۸۱.۲۰٪ رسید. تحلیل علت این افت نشان داد که تبدیل وزن‌ها به ۸ بیت موجب ایجاد نویز در محاسبات attention heads در مدل BERT شد و حساسیت مدل به سیگنال‌های ظریف متنی کاهش یافت. همچنین، استفاده از مجموعه کوچک validation در benchmark می‌تواند اثر rounding را تقویت کرده و کاهش دقت را بزرگ‌تر نشان دهد.

در جمع‌بندی، Quantization و Batching یک موفقیت بزرگ در کاهش حجم و افزایش سرعت مدل به همراه داشت، اما افت دقت ۸.۴٪ نشان داد که برای استفاده در محیط عملیاتی ممکن است نیاز به تنظیمات و بررسی دقیق نمونه‌ها باشد تا تجربه کاربری تحت تأثیر قرار نگیرد.